

MongoDB Health Check — Key Steps for Optimal Performance

When performing a **MongoDB health check**, I follow these key steps to ensure optimal stability, data consistency, and performance:

1. Verify MongoDB Services

- Confirm mongod, mongos, and config server processes are running.
- Check service status and startup configuration on all nodes.

2. Review MongoDB Logs

- Inspect the mongod.log for warnings, slow queries, replication lag, or connection errors.
- Review system logs for OS-level issues impacting MongoDB.

3. Check Cluster & Replica Set Status

- Run `rs.status()` to verify primary/secondary state, replication health, and member sync status.
- Ensure there is no replication lag or election instability.

4. Assess Sharded Cluster Health (if applicable)

- Run `sh.status()` to confirm all shards, config servers, and mongos routers are connected.
- Validate chunk distribution and balancer state.

5. Monitor Resource Utilization

- Review CPU, memory, disk I/O, and network metrics.
- Identify high resource consumption or swapping.

6. Evaluate Storage & Disk Usage

- Monitor data directory size and free space.
- Verify journaling is enabled and disk latency is within acceptable limits.

7. Check Database and Collection Statistics

- Use `db.stats()` and `db.collection.stats()` to review storage size, index size, and document count.
- Identify collections with abnormal growth.

8. Index and Query Performance

- Use `explain()` to analyze query execution plans.
- Detect missing or unused indexes and long-running operations via `db.currentOp()`.

9. Replication & Oplog Health

- Validate oplog size and ensure sufficient retention window.
- Check replication delay and heartbeat latency between nodes.

10. Backup & Restore Validation

- Confirm that backup jobs (mongodump, snapshot, or cloud backups) are recent and restorable.
- Test restore to staging environment periodically.

11. Security & Access Review

- Verify authentication (SCRAM-SHA), user roles, and privileges.
- Review IP bindings, TLS/SSL setup, and audit logs for failed access attempts.

12. Configuration Review

- Validate key parameters like `wiredTigerCacheSizeGB`, `storageEngine`, `replication.oplogSizeMB`, etc.
- Check for any deviations from baseline configurations.

13. Monitor Alerts & Metrics

- Review dashboards in MongoDB Atlas, CloudWatch, or custom Prometheus/Grafana setups.
- Ensure alert thresholds (replication lag, disk space, memory) are tuned appropriately.

14. Run Validation & Consistency Checks

- Use db.collection.validate() for data integrity.
- Verify schema consistency if using schema validation or MongoDB Compass.

15. Document Findings & Actions

- Record all checks, detected issues, and corrective actions.
- Update runbooks and health check logs for future audits.